

Study Agent Project Report

An AI study platform for uploading PDFs, extracting topic knowledge, assigning document access, and allowing students or researchers to chat with the material without receiving the original PDF file.

[Next.js Frontend](#)[Express Backend](#)[MongoDB](#)[S3 PDF Storage](#)[Backend MCP Routes](#)[RAG Chat](#)

Purpose: Explain what is being built, what happens inside the system, the main features, and how researchers/teachers can assign PDFs to students while keeping access controlled.

Maximum length: Designed as an 8 page report for presentation or submission.

1. Project Overview

Study Agent is a document-centered AI learning application. A user uploads a PDF such as notes, textbook chapters, research papers, or academic material. The backend stores the original PDF in S3, extracts text, builds a hierarchy of topics and subtopics, stores those topics in MongoDB, and makes them available to the chat and study interfaces.

The important idea is that the chat is not a generic chatbot. When a chat is opened inside a document, the backend builds a prompt from that document's extracted topics, weak areas, and relevant content chunks. This makes the assistant answer from the student's current material instead of from an unrestricted general context.

For Students

- Upload study PDFs and organize them by document.
- Ask questions in a file-specific chat.
- Explore topics through mind maps and flash cards.
- Receive suggestions based on weak or unattempted topics.
- Track performance when the chat evaluates understanding.

For Researchers and Teachers

- Upload a paper, chapter, or curated reading material.
- Assign access to students by email.
- Let students ask questions from the material.
- Avoid sending the raw PDF file directly.
- Revoke access later if required.

Core Value

Study Agent turns static PDFs into interactive learning sessions. Instead of only reading a document, students can ask, revise, test themselves, visualize the topic structure, and maintain a chat history per document.

Problem	Study Agent Solution
Students struggle to navigate long PDFs.	PDFs are converted into topic hierarchy, mind map, summaries, and flash cards.
Generic AI answers may drift away from course material.	Chats are scoped using extracted document topics and relevant content chunks.
Teachers need to distribute material but may not want to share the original file.	Assignment grants app access for chat and study views while raw download is owner-only.
Students do not know what to revise next.	Weak topics and not-attempted topics are surfaced for study planning.

2. Core Functionalities

The application is built around four connected workflows: document upload, topic extraction, document-scoped chat, and performance tracking.

PDF Upload and Storage

Users upload PDF files through the app. The backend validates that the file is a PDF, stores it in AWS S3, records metadata in MongoDB, and computes a SHA256 hash for deduplication.

Backend /api/files/upload

Automatic Topic Extraction

After upload, the backend starts background PDF processing. It extracts text, detects document structure, generates summaries, creates embeddings, and stores topics recursively.

Service pdfHierarchyService.js

File-Specific Chat

Each PDF can have multiple chats. When a student chats inside a file, the backend loads the document's topics and relevant chunks before generating the answer.

API /api/chat-stream

Assignment by Email

Owners can assign a PDF to other users by email. Assigned users see the document in their sidebar and can create chats, while delete and owner-only download are not available to them.

API /api/files/:id/assign

Topic Explorer

Students can view the extracted topic hierarchy as a mind map, search the map, open topic details, and use flash cards generated from topics.

Frontend /topics/[fileId]

Records and Weak Topics

The system saves topic-level marks when chat interactions show student understanding. Records show overall score, attempts, weak topics, and learning progress.

Data FolderSchema

What Makes It Different

- The PDF becomes a structured knowledge base, not just a file upload.
- Chats are organized by file, so learning context stays clean.
- Assigned documents appear separately from owned documents.
- The app supports both study assistance and assessment-style tracking.
- Backend MCP routes expose study context to AI/tool workflows in a controlled way.

3. System Architecture

The system has two active application layers: the Next.js frontend and the Express backend. MCP support is implemented inside the backend through internal guarded routes, rather than through the separate `mcp` folder. MongoDB stores users, files, topics, chats, progress, and performance records. AWS S3 stores the original PDFs. AI models are used for document analysis, chat responses, title generation, summaries, embeddings, and score detection.



Frontend Responsibilities

- Shows landing, authentication, home, chat, file dashboard, topic explorer, records, suggestions, and profile screens.
- Stores the auth token in browser storage and sends it with API requests.
- Streams chat responses from the backend using Server-Sent Events.
- Organizes sidebar items into General chats, My Files, and Assigned to Me.
- Allows file owners to assign and revoke document access.

Backend Responsibilities

- Connects to MongoDB and registers protected API routes.
- Validates uploaded PDFs and sends them to S3.
- Starts background processing to extract topics from PDFs.
- Builds study-context prompts and streams LLM responses.
- Saves chat history, chat sessions, rolling summaries, scores, and records.
- Protects backend MCP/internal routes using the `x-mcp-secret` header.

Backend MCP Responsibilities

- Implements MCP-style context access in the backend instead of relying on the separate `mcp` folder.
- Provides internal endpoints for folder context, weak topics, and RAG context.
- Uses a shared secret so outside clients cannot call internal context routes directly.
- Keeps study context and database access centralized in the backend.

4. PDF Upload and Processing Flow

When a user uploads a PDF, the backend does more than store the file. It prepares the PDF for AI-powered study by extracting text and transforming it into structured topics.

1

Upload

The frontend sends the PDF to the backend. Multer stores it temporarily in memory and rejects non-PDF files.

2

Hash and Name

The backend computes a SHA256 hash to detect duplicates. It reads the first page and derives a cleaner dynamic file name.

3

Deduplication

If the same PDF hash already exists, the system creates a new file entry and copies existing topics instead of reprocessing the document.

4

S3 Storage

If it is a new PDF, the backend uploads the original file to S3 and stores the S3 key in MongoDB.

5

Background Processing

The backend runs `processPDFHierarchy`. Progress is saved in `ProcessingProgress` so the frontend can poll status.

6

Extraction and Analysis

The processor tries `pdfjs-dist`, then PyMuPDF, then OCR fallback for scanned PDFs. It analyzes document type, topics, summaries, and embeddings.

7

Topic Storage

Main topics and child topics are recursively saved into the `Topic` collection with level, parent topic ID, summary, content, and order.

Topic Data Model

Each topic belongs to a file and user. It includes hierarchy level, optional parent topic, title, summary, content, embedding, page metadata, order, and child count. This structure powers the mind map, flash cards, RAG retrieval, and chat grounding.

Progress Tracking

The frontend polls `/api/topics/progress/:fileId` every few seconds. The dashboard and topic page can show whether a document is pending, processing, completed, or failed.

5. Chat, RAG, and Learning Records

The chat system is built around document context. When a user opens a chat with a `fileId`, the backend associates the chat session with that file. Future requests can recover the file context from the chat session if the frontend does not send it again.

How a Chat Answer Is Generated

1. The frontend posts the user message, chat ID, and optional file ID to `/api/chat-stream`.
2. The backend loads or creates `ChatContext` for rolling memory.
3. If a file is attached, the backend loads all topics, performance marks, weak topics, and relevant chunks.
4. The backend builds a system prompt that restricts the assistant to the student's study material.
5. The LLM response is streamed token by token to the frontend using SSE.
6. The backend saves the user message and assistant response in `ChatMessage`.
7. The backend updates the chat title and rolling conversation summary.

RAG Context

The app currently retrieves relevant topic chunks by matching the query against topic titles, summaries, and content. The topic model already stores embeddings, so this can later be upgraded to vector search for stronger semantic retrieval.

Performance Scoring

After a file-scoped chat response, the backend asks the model whether the exchange demonstrated understanding of a topic. If yes, it extracts a topic name, gives a score from 0 to 10, converts it to a percentage, and stores it in `FolderSchema`. The frontend receives a score update event and can show a toast.

Stored Item	Purpose
<code>ChatSession</code>	Stores chat title, file ID, folder ID, file name, user ID, and timestamps.
<code>ChatMessage</code>	Stores user and assistant messages for paginated chat history.
<code>ChatContext</code>	Stores recent messages and rolling summary to keep conversation memory compact.
<code>FolderSchema</code>	Stores marks, average performance score, weak flag, and topic performance records.

Student Experience

The student sees a chat interface with streaming responses, copy, resend, edit-message, title rename, chat history pagination, and score notifications. Chats can be general or attached to a specific PDF.

6. Assignment and Controlled PDF Access

The assignment feature lets a file owner grant another user access to a PDF-based study experience by email. This is useful for teachers, research mentors, and project leads who want students to learn from a document without sending the original PDF file directly.

How Assignment Works

1 Owner Opens File Dashboard

The dashboard shows the selected PDF, processing progress, chats, topics link, and an Assign button.

2 Owner Enters Emails

The owner can enter one or multiple emails separated by commas or spaces.

3 Backend Updates Access List

The backend verifies the requester owns the file, then appends emails to the file's `assignedTo` array.

4 Email Notification

The backend attempts to send an assignment email to each new recipient.

5 Assigned User Sees the File

The file listing route returns files where the user is owner or where their email appears in `assignedTo`.

6 Assigned User Chats

The assigned user can create chats for that file and ask questions grounded in its topics.

Access Rules in Current Code

Action	Owner	Assigned User
List file	Allowed	Allowed if email is assigned
Create chat for file	Allowed	Allowed
Open topic dashboard	Allowed by file visibility	Allowed by file visibility in UI flow
Assign or revoke users	Allowed	Not shown in frontend and backend requires ownership
Delete file	Allowed	Not shown in frontend and backend requires ownership
Download original PDF	Allowed through owner-only route	Blocked by owner check

Important content-protection note: The current system does not share the original PDF file with assigned users, but assigned users can still learn information from extracted topics and chat answers. If the goal is strict non-disclosure of document content, the topic explorer should be restricted and chat responses should be configured to answer only at a limited abstraction level.

Why This Helps Researchers and Teachers

- They can assign a paper, chapter, or reading set without emailing the PDF.
 - Students can ask questions and understand the material interactively.
 - The owner can revoke access when the assignment period ends.
 - Students' interaction can create performance records and weak-topic signals.
-
- The same processed PDF can be reused through deduplication, reducing repeated processing cost.

7. MCP, Backend, and Frontend Details

Backend MCP Tools

MCP is implemented in the backend only. The separate `mcp` folder is not treated as an active runtime component in this report. Backend routes expose guarded study-context operations using the shared `MCP_SECRET`, keeping database logic centralized in the Express application.

Backend MCP Operation	What It Returns	Why It Matters
<code>get_folder_context</code>	All extracted topics and performance marks for a file.	Lets an AI understand what material exists in the current study session.
<code>get_weak_topics</code>	Topics below a performance threshold or not attempted.	Supports personalized revision suggestions.
<code>get_rag_context</code>	Relevant topic chunks for a query.	Grounds answers in actual document content.

Backend Modules

- `index.js` : starts Express, CORS, JSON parsing, MongoDB connection, public API routes, and backend MCP/internal routes.
- `fileRoutes.js` : handles PDF upload, file listing, download URL, rename, delete, assignment, and revoke.
- `topicRoutes.js` : starts or checks processing and returns topic hierarchy.
- `chatStreamController.js` : builds system prompt, streams model output, saves chat messages, updates scores, and manages memory.
- `studyContextService.js` : fetches topics, weak topics, and RAG chunks directly from MongoDB for chat prompting.
- `internalMcpRoutes.js` : exposes backend context endpoints only to trusted callers with the secret header.

Frontend Modules

- `/home` : starts general chats with suggested prompts.
- `/chat/[id]` : streams chat, loads history, initializes file context, shows score updates, and supports message tools.
- `/dashboard/[fileId]` : shows document-level chats, processing progress, topics link, assignment modal, and revoke list.
- `/topics/[fileId]` : renders mind map and flash card views from extracted topics.
- `Sidebar.tsx` : organizes chats and files into General, My Files, and Assigned to Me sections.
- `/records` : summarizes scores and attempts across owned and assigned files.

Security Controls Present

- Protected user API routes require JWT verification.
- Assignment and revoke actions require file ownership.
- Original PDF download route checks ownership.
- Backend MCP/internal routes require a shared secret header.
- S3 file access is through presigned URL generation, not public file URLs.

8. Impact, Use Cases, and Future Improvements

Use Cases

- **Classroom study:** A teacher uploads chapter PDFs and assigns them to students for guided chat-based revision.
- **Research onboarding:** A researcher assigns papers to juniors so they can ask conceptual questions without receiving the raw PDF.
- **Exam preparation:** Students upload notes, generate topic maps, quiz themselves, and focus on weak areas.
- **Collaborative reading:** A group can work from the same assigned source while each user's chats remain separate.

Recommended Improvements

Area	Improvement
Strict content protection	Add permissions to hide topic content from assigned users or provide only chat access with controlled answer detail.
Retrieval quality	Replace regex-based RAG with vector search using the stored embeddings.
Authorization consistency	Make topic routes explicitly allow both owners and assigned users, instead of relying on user-owned topic queries only.
Auditability	Log assignment, revoke, chat, and download events for researchers or institutions.
Analytics	Add teacher-facing dashboards showing student engagement, weak areas, and completion.
Scalability	Move PDF processing into a job queue so large PDFs do not depend on long-running request-side background work.

Conclusion

Study Agent is a practical AI learning system that converts PDFs into structured, interactive study material. The backend handles storage, processing, topic creation, chat grounding, scoring, assignment, and MCP-style internal context access. The frontend gives users an organized study workspace with chat, dashboards, mind maps, flash cards, records, and controlled access flows.

The strongest product story is controlled learning access: a researcher or teacher can upload a PDF, assign it to students, and let them chat with the knowledge inside the app without directly distributing the original PDF file. With stricter content controls and vector retrieval, this can become a strong platform for academic study, research onboarding, and personalized revision.

One-Line Summary

Study Agent transforms uploaded PDFs into secure, assigned, AI-powered study sessions where students can ask questions, explore topics, and track learning progress.